Práctica Integral: Clasificación con Validación Estadística

Análisis de Tráfico de Red y Detección de Intrusos usando CIC-IDS2017

Christian Gómez Magdaleno
Noe Israel Felipe Perez
Andrea Guadalupe Garcia Oñate
Universidad de Guanajuato

13 de abril de 2026

Resumen

En el presente trabajo se desarrolla una solución completa de clasificación multiclase orientada a la ciberseguridad, utilizando el conjunto de datos CIC-IDS2017. El objetivo principal es evaluar y comparar el rendimiento de dos algoritmos (XGBoost y Random Forest) mediante un diseño experimental basado en validación cruzada estratificada repetida. Se aplican pruebas de hipótesis estadísticas para determinar la significancia de las diferencias observadas.

1. Definición del Problema

El conjunto de datos utilizado es el **CIC-IDS2017** (Canadian Institute for Cybersecurity), el cual contiene capturas de tráfico de red benigno y los ataques cibernéticos más actualizados. Tras un proceso de limpieza y auditoría que mitigó la multicolinealidad, valores nulos, varianza cero e inconsistencias generadas por el extractor de características, el conjunto de datos final cuenta con atributos numéricos.

El problema se enmarca como una **clasificación multiclase**, donde la variable objetivo (*Label*) busca identificar si el tráfico es legítimo o si pertenece a una categoría de ataque específica.

Contexto e Impacto: La decisión operativa que se apoya con este modelo es la gestión automatizada del tráfico y la generación de alertas en un Sistema de Detección de Intrusos (IDS) en tiempo real. En este dominio un *Falso Positivo* (bloquear a un usuario legítimo) genera fricción operativa temporal, pero un *Falso Negativo* (no detectar un ataque) puede comprometer la confidencialidad, integridad y disponibilidad de una infraestructura crítica.

Abordar este problema mediante una **clasificación multiclase** nos permite representar de mejor manera la realidad y es fundamental para las operaciones de un

Centro de Operaciones de Seguridad (SOC). Identificar con precisión la topología del ataque permite:

- **Respuesta a Incidentes Diferenciada:** Las estrategias de mitigación varían según la amenaza. Un ataque de Denegación de Servicio (DDoS) requiere reglas de enrutamiento y balanceo de carga en la red, mientras que un ataque de Fuerza Bruta exige bloqueos de IP a nivel de firewall y rotación de credenciales.
- **Priorización (Triage):** No todos los ataques tienen la misma urgencia. Un escaneo de puertos (*PortScan*) suele ser ruido exploratorio de baja prioridad, pero una infiltración de red (*Infiltration*) o un ataque a aplicaciones web (*Web Attack*) representan brechas activas que requieren atención humana inmediata.
- **Inteligencia de Amenazas y Análisis Forense:** Etiquetar correctamente el tipo de ataque permite a los equipos defensivos entender las Tácticas, Técnicas y Procedimientos del adversario, facilitando el parcheo de la vulnerabilidad raíz (ej. reparar una base de datos tras detectar inyecciones SQL).

2. Selección y Justificación de Métricas

El dataset **CIC-IDS2017**, tiene un desbalance de clases masivo. Bajo este escenario, la métrica de Exactitud (*Accuracy*) resulta engañosa, un modelo podría ignorar todas las clases minoritarias y aún así obtener un puntaje cercano al 100 %, fallando en su propósito fundamental de detección.

Para mitigar este sesgo, se ha seleccionado el **F1-Score Macro** como métrica primaria

2.1. F1-Score Macro

El F1-Score es la media armónica entre la precisión y la sensibilidad. A diferencia de la media aritmética, la armónica penaliza valores extremos, asegurando que un modelo deba tener un buen desempeño en ambas métricas para obtener un F1 alto:

$$F1_i = 2 \cdot \frac{Precision_i \cdot Recall_i}{Precision_i + Recall_i}$$

Para un problema de clasificación multiclase con N categorías, existen dos variantes principales: *Micro* (ponderada por el número de instancias) y *Macro* (promedio simple). En esta práctica se utiliza el enfoque **Macro**, definido formalmente como:

$$F1_{Macro} = \frac{1}{N} \sum_{i=1}^{N} F1_i$$

Justificación: Al calcular el promedio no ponderado de los $F1$ individuales, tratamos a cada categoría de ataque con la misma importancia, independientemente de su frecuencia. Esto es crítico en ciberseguridad, ya que un ataque de Infiltración es extremadamente raro pero potencialmente devastador para la infraestructura; el F1-Macro garantiza que el modelo sea penalizado severamente si falla en detectar estas clases minoritarias, obligando a los algoritmos (XGBoost y Random Forest) a aprender fronteras de decisión representativas para todo el espectro de amenazas.

3. Selección de Modelos

Para abordar la complejidad dimensional y las relaciones no lineales del tráfico de red, se seleccionaron dos arquitecturas basadas en ensambles de árboles:

3.1. XGBoost (Extreme Gradient Boosting)

XGBoost es una implementación optimizada del algoritmo de *Gradient Boosting*. A diferencia de los métodos de ensamble paralelo, XGBoost construye árboles de forma secuencial. Cada nuevo árbol intenta predecir los errores residuales (la diferencia entre el valor real y la predicción acumulada) de los árboles anteriores.

Detalle Técnico y Supuestos:

- **Función Objetivo:** Minimiza una función de pérdida que incluye un término de regularización L1 y L2 para evitar el sobreajuste:

$$\mathcal{L}(\phi) = \sum_i l(\hat{y}_i, y_i) + \sum_k \Omega(f_k)$$

  donde $\Omega(f) = \gamma T + \frac{1}{2}\lambda \|w\|^2$ penaliza la complejidad del modelo.

- **Ventajas:** Manejo nativo de valores faltantes, soporte para computación distribuida y GPU, y poda de árboles basada en ganancia negativa.

Algoritmo 1: Pseudocódigo de XGBoost

Entrada: Dataset $\mathcal{D}$, Función de pérdida L, Tasa de aprendizaje η
Salida: Modelo Final $F(x)$

```
Inicializar F_0(x) = arg mín_γ Σ_{i=1}^{n} L(y_i, γ);
for m = 1 to M do
    Calcular gradientes y hessianos de la función de pérdida;
    Entrenar un árbol base f_m(x) usando los residuos del modelo F_{m-1}(x);
    Actualizar el modelo: F_m(x) = F_{m-1}(x) + η · f_m(x);
end
```

3.2. Random Forest

Es un algoritmo de aprendizaje supervisado que utiliza la técnica de *Bagging* (Bootstrap Aggregating). Construye un "bosque"de árboles de decisión no correlacionados entrenados en diferentes subconjuntos de datos.

Detalle Técnico y Supuestos:

- **Inyección de Aleatoriedad:** Además de usar muestras aleatorias de filas (*Bootstrapping*), selecciona un subconjunto aleatorio de características en cada división de nodo. Esto reduce la varianza del modelo sin aumentar significativamente el sesgo.

- **Ventajas:** Excelente para manejar el desbalance de clases (usando `class_weight`) y proporciona una medida intrínseca de la importancia de las variables.

- **Supuestos:** Se asume que los árboles individuales tienen suficiente profundidad para capturar patrones, confiando en el promedio para suavizar el error.

Algoritmo 2: Pseudocódigo de Random Forest

Entrada: Dataset $\mathcal{D}$, Número de árboles B
Salida: Ensamble de árboles $G(x)$

```
for b = 1 to B do
    Tomar una muestra Bootstrap D_b del dataset original;
    Entrenar un árbol T_b sobre D_b;
        En cada nodo, seleccionar aleatoriamente k características de las p
          totales;
        Elegir la mejor división entre las k características;
end
Predicción: Promediar las probabilidades (regresión) o votación
  mayoritaria (clasificación);
```

4. Pruebas Estadísticas

4.1. Prueba de Normalidad (Shapiro-Wilk)

La prueba de **Shapiro-Wilk** calcula un estadístico W, el cual mide la correlación entre los datos observados y los valores teóricos de una distribución normal:

$$W = \frac{\left(\sum_{i=1}^{n} a_i x_{(i)}\right)^2}{\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

Donde $x_{(i)}$ son los datos ordenados y a_i son coeficientes basados en los valores esperados de los estadísticos de orden de una distribución normal. Un valor de W cercano a 1,0 indica una alta adherencia a la normalidad.

Planteamiento de Hipótesis:

- H_0: Las diferencias de rendimiento siguen una distribución normal ($p > 0{,}05$).
- H_1: Las diferencias de rendimiento **no** siguen una distribución normal ($p \leq 0{,}05$).

Esta prueba actúa como un filtro metodológico: si se acepta H_0, se utiliza la *Prueba t de Student pareada*; de lo contrario, se opta por la prueba no paramétrica de *Wilcoxon*, garantizando que las conclusiones del estudio sean estadísticamente válidas.

4.2. Prueba t de Student Pareada

En caso de que el supuesto de normalidad sea satisfecho, se emplea la **Prueba t de Student pareada**. La prueba evalúa si la media de las diferencias ($\bar{d}$) entre los dos modelos es significativamente distinta de cero. El estadístico t se calcula como:

$$t = \frac{\bar{d}}{s_d/\sqrt{n}}$$

Donde:

- $\bar{d}$: Es el promedio de las diferencias entre los puntajes de XGBoost y Random Forest.

- s_d: Es la desviación estándar de esas diferencias.
- n: Es el número total de ejecuciones ($n = 30$).

El valor de t se contrasta con una distribución teórica con $n-1$ grados de libertad para obtener el p-value.

Interpretación:

- Si $p \leq 0{,}05$: Se rechaza H_0, concluyendo que un modelo es superior al otro con un nivel de confianza del 95 %.
- Si $p > 0{,}05$: Se concluye que las diferencias observadas son producto del azar y no existe superioridad estadística entre los algoritmos.

5. Diseño Experimental

5.1. Partición de Datos

Previo a cualquier fase de entrenamiento, el conjunto de datos fue dividido de forma estratificada. Se reservó un **15 % como conjunto de prueba (*Test Set*)** completamente aislado, el cual fungió como evaluación final sobre datos nunca antes vistos. Sobre el 85 % restante, se llevó a cabo el proceso de validación cruzada (80 % entrenamiento y 20 % validación).

5.2. Preprocesamiento y Escalado

Todas las variables numéricas fueron normalizadas utilizando **Estandarización (*StandardScaler*)**. Este proceso ajustó la distribución de cada variable para tener una media de 0 y una desviación estándar de 1, estabilizando el proceso de aprendizaje matemático.

5.3. Validación Cruzada Estratificada Repetida

Sobre el conjunto de entrenamiento, se implementó un diseño de **Validación Cruzada Estratificada Repetida** (*Repeated Stratified K-Fold*). Se utilizó una configuración de $k = 5$ particiones repetidas 6 veces, generando un total de **30 ejecuciones independientes** por modelo. La estratificación aseguró que en cada iteración se mantuviera la proporción exacta de las clases originales, permitiendo obtener distribuciones de rendimiento estables y comparables para el análisis estadístico posterior.

5.4. Ajuste de Hiperparámetros

Se llevó a cabo una búsqueda empírica de hiperparámetros con el objetivo de configurar los modelos para que alcanzaran un desempeño óptimo y una generalización adecuada.

Tabla 1: Configuración de hiperparámetros

Modelo	**Hiperparámetro**	**Valor Final**
XGBoost	*n_estimators*	150
	learning_rate (η)	0.08
	max_depth	6
	subsample	0.8
	colsample_bytree	0.8
	max_delta_step	1
	reg_alpha ($L1$)	1
	reg_lambda ($L2$)	2.5
	gamma	0.2
Random Forest	*n_estimators*	200
	max_depth	17
	class_weight	balanced
	min_samples_split	10
	min_samples_leaf	4

6. Análisis Estadístico

Se aplicó la prueba de **Shapiro-Wilk** para evaluar el supuesto de normalidad. Dado que los datos presentaron una distribución normal ($p = 0{,}3403 > 0{,}05$), se procedió a utilizar la **Prueba t de Student pareada**.

Los resultados arrojaron un estadístico de 15,7672 y un valor $p = 9{,}2141 \times 10^{-16}$, lo cual es significativamente menor que el nivel de significancia $\alpha = 0{,}05$. En consecuencia, se **rechaza la hipótesis nula (H_0)**, concluyendo que existe una diferencia estadísticamente significativa entre los modelos

7. Evaluación de Resultados

A continuación, se presentan las métricas descriptivas extraídas tras la ejecución del diseño experimental para la métrica F1-Score Macro a lo largo de 30 iteraciones.

Tabla 2: Resultados detallados de F1-Score (Macro) por ejecución

	XGBoost			Random Forest		
Ej.	**Train**	**Val**	**Test**	**Train**	**Val**	**Test**
1	0.8825	0.8249	0.8246	0.7890	0.7375	0.7530
2	0.8453	0.8269	0.8082	0.7679	0.7576	0.7358
3	0.8499	0.8346	0.8152	0.7574	0.7109	0.7298
4	0.8608	0.8262	0.8118	0.8313	0.8061	0.7556
5	0.8512	0.8082	0.8101	0.7745	0.7360	0.7442
6	0.8449	0.7874	0.7943	0.7666	0.7226	0.7348
7	0.8530	0.8173	0.8031	0.7661	0.7519	0.7353
8	0.8816	0.8298	0.8359	0.7733	0.7580	0.7429
9	0.8844	0.8712	0.8325	0.8275	0.7871	0.7509
10	0.8488	0.8160	0.8013	0.8454	0.7491	0.7684
11	0.8889	0.8374	0.8297	0.7881	0.7486	0.7495
12	0.8898	0.8651	0.8307	0.8521	0.7939	0.7957
13	0.8891	0.8544	0.8240	0.7991	0.7865	0.7556
14	0.8700	0.8275	0.8274	0.7819	0.7300	0.7326
15	0.8487	0.8233	0.8065	0.7761	0.7460	0.7407
16	0.8781	0.8565	0.8265	0.8043	0.7561	0.7526
17	0.8472	0.8196	0.8076	0.8258	0.7803	0.7712
18	0.8560	0.8342	0.8194	0.7826	0.7442	0.7429
19	0.8668	0.8219	0.8127	0.7712	0.7357	0.7395
20	0.8628	0.8026	0.8206	0.8140	0.7848	0.7426
21	0.8453	0.8000	0.8052	0.7625	0.7271	0.7347
22	0.8491	0.8205	0.7923	0.7799	0.7525	0.7376
23	0.8929	0.8282	0.8383	0.7915	0.7600	0.7497
24	0.8538	0.8338	0.8159	0.7860	0.7384	0.7459
25	0.8862	0.8348	0.8118	0.8185	0.7827	0.7489
26	0.8842	0.8406	0.8208	0.7760	0.7370	0.7331
27	0.8901	0.8784	0.8254	0.7847	0.7471	0.7441
28	0.8421	0.8447	0.7987	0.7766	0.7474	0.7390
29	0.8783	0.8383	0.8219	0.8206	0.8046	0.7588
30	0.8935	0.8198	0.8285	0.7637	0.7139	0.7318

Tabla 3: Estadísticas descriptivas del F1-Score (Macro) en Validación tras 30 iteraciones

Métrica Estadística	**XGBoost**	**Random Forest**
Mínimo	0.7874	0.7109
Máximo	0.8784	0.8061
Mediana	0.8279	0.7489
Promedio (μ)	0.8308	0.7545
Desviación Estándar (σ)	0.0199	0.0252

8. Visualización

Para apoyar el análisis descriptivo, se ilustra la dispersión y estabilidad de ambos algoritmos mediante diagramas de cajas y rankings de error.

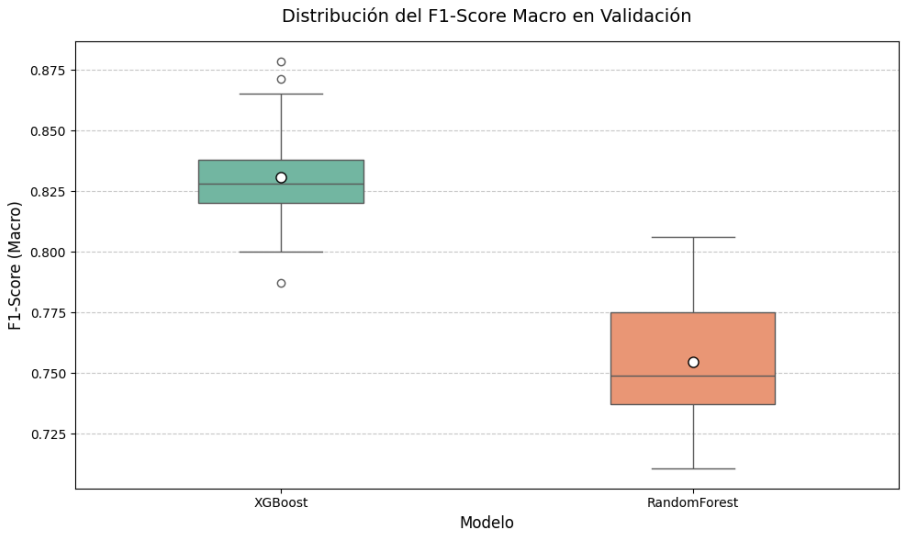

Figura 1: Boxplot de la distribución del F1-Score Macro a través de las 30 iteraciones.

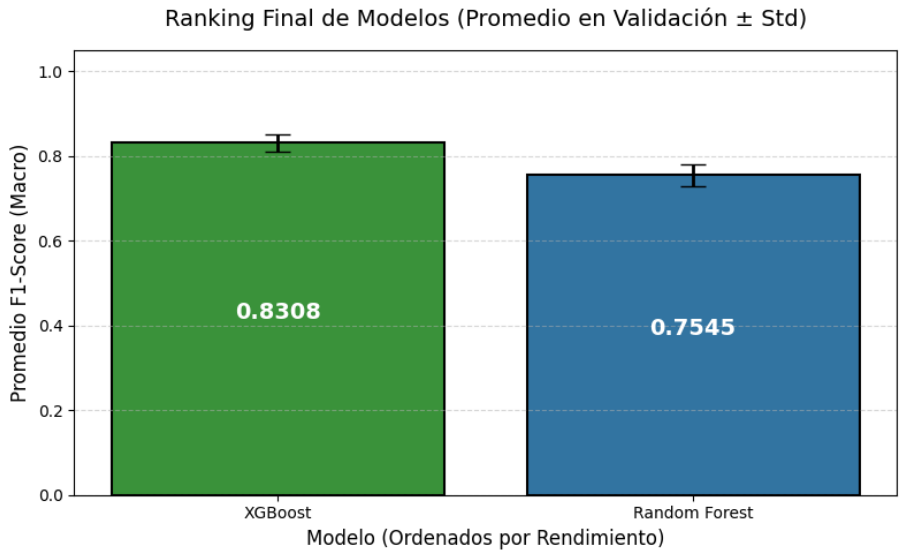

Figura 2: Ranking promedio de modelos con márgenes de error (Desviación Estándar).

.

9. Análisis Detallado por Modelo

9.1. XGBoost

A continuación, se presentan los resultados detallados de la mediana del desempeño global en validación ($F1 \approx 0{,}8279$), correspondiente a la mediana en validación.

Tabla 4: Reporte de clasificación detallado: XGBoost

Clase	**Precision**	**Recall**	**F1-Score**	**Support**
BENIGN	1.00	1.00	1.00	340,881
Bot	0.42	0.99	0.59	295
DDoS	1.00	1.00	1.00	19,204
DoS GoldenEye	0.96	1.00	0.98	1,544
DoS Hulk	1.00	1.00	1.00	34,519
DoS Slowhttptest	0.89	0.99	0.94	825
DoS slowloris	0.98	0.99	0.99	869
FTP-Patator	1.00	1.00	1.00	1,191
Heartbleed	1.00	1.00	1.00	2
Infiltration	0.60	0.60	0.60	5
PortScan	0.99	1.00	1.00	23,839
SSH-Patator	0.95	1.00	0.97	885
Web Attack – Brute Force	0.71	0.65	0.68	226
Web Attack – Sql Injection	0.22	0.67	0.33	3
Web Attack – XSS	0.25	0.52	0.34	98
Macro Average	**0.80**	**0.89**	**0.83**	**424,386**
Weighted Average	**1.00**	**1.00**	**1.00**	**424,386**

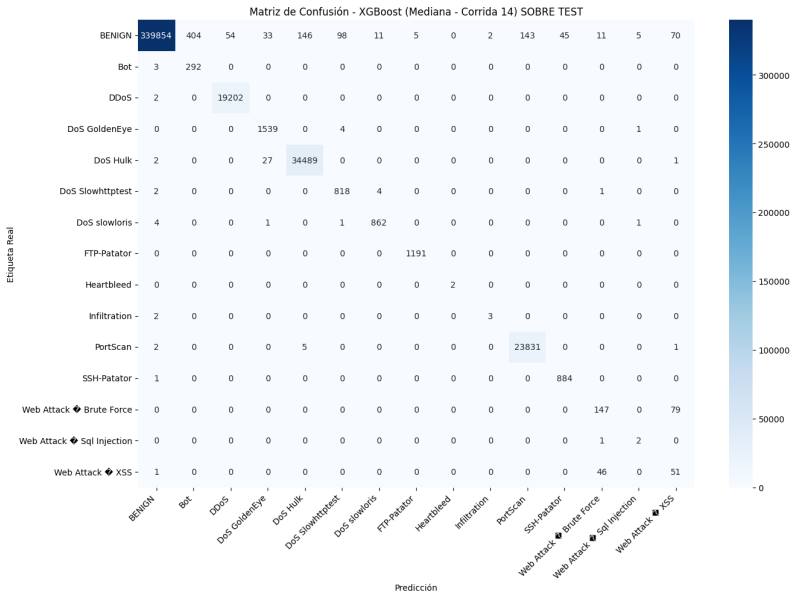

Figura 3: Matriz de Confusión: XGBoost Mediana en Validación

Tabla 5: Desempeño Global F1-Score (Macro) - XGBoost

Conjunto de Datos	**F1-Score (Macro)**
Entrenamiento (Train)	0.8700
Validación (Val)	0.8275
Prueba (Test)	0.8274

9.2. Random Forest

A continuación, se presentan los resultados detallados de la mediana del desempeño global en validación ($F1 \approx 0{,}7491$), correspondiente a la mediana en validación.

Tabla 6: Reporte de clasificación detallado: Random Forest

Clase	**Precision**	**Recall**	**F1-Score**	**Support**
BENIGN	1.00	0.98	0.99	340,881
Bot	0.07	0.99	0.13	295
DDoS	0.99	1.00	1.00	19,204
DoS GoldenEye	0.97	1.00	0.98	1,544
DoS Hulk	0.93	1.00	0.96	34,519
DoS Slowhttptest	0.94	0.99	0.96	825
DoS slowloris	0.99	0.99	0.99	869
FTP-Patator	0.99	1.00	0.99	1,191
Heartbleed	1.00	1.00	1.00	2
Infiltration	1.00	0.20	0.33	5
PortScan	0.99	1.00	1.00	23,839
SSH-Patator	0.92	1.00	0.96	885
Web Attack – Brute Force	0.52	0.46	0.49	226
Web Attack – Sql Injection	0.50	0.67	0.57	3
Web Attack – XSS	0.09	0.86	0.17	98
Macro Average	**0.79**	**0.87**	**0.77**	**424,386**
Weighted Average	**0.99**	**0.98**	**0.99**	**424,386**

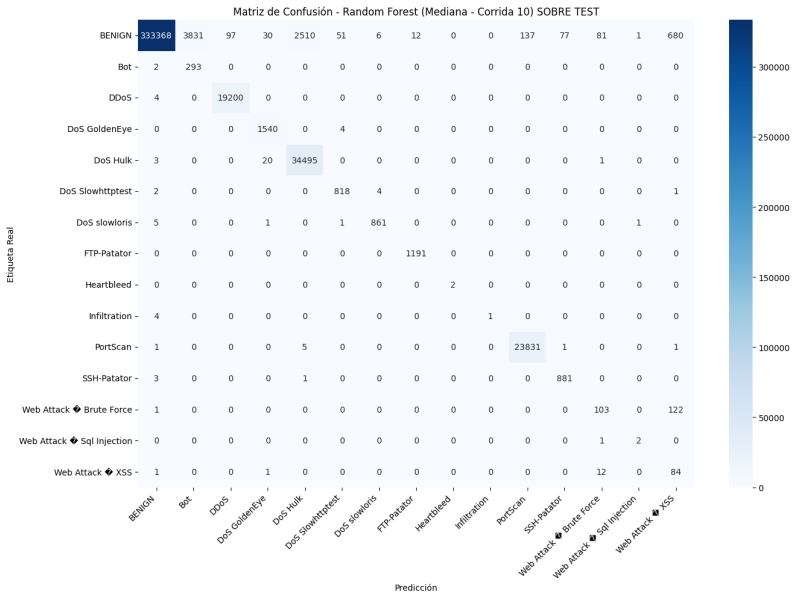

Figura 4: Matriz de Confusión: Random Forest Mediana en Validación

Tabla 7: Desempeño Global F1-Score (Macro)

Conjunto de Datos	**F1-Score (Macro)**
Entrenamiento (Train)	0.8454
Validación (Val)	0.7491
Prueba (Test)	0.7684

10. Discusión

El análisis de los resultados revela un fenómeno de sobreajuste (overfitting) en ambos modelos, manifestado en la brecha entre el desempeño de entrenamiento y los resultados de validación y prueba. Este comportamiento se fundamenta principalmente en la naturaleza del conjunto de datos y la sensibilidad de las métricas seleccionadas.

10.1. El Impacto del Desbalance de Clases y el F1-Macro

El conjunto de datos presenta un desbalance extremo: la clase *BENIGN* representa aproximadamente el 80 % del total con 2.2 millones de registros, mientras que categorías críticas como *Heartbleed*, *SQL Injection* e *Infiltration* cuentan apenas con 11, 21 y 36 registros respectivamente.

Al utilizar el **F1-Score Macro** como métrica principal, se otorga el mismo peso estadístico a cada una de las 15 clases sin considerar su soporte. En este escenario, el modelo logra aprender con facilidad los patrones de la clase mayoritaria (alcanzando un F1 cercano a 1.0), pero falla al generalizar en clases minoritarias. El sobreajuste observado no implica necesariamente una memorización del ruido general, sino una memorización de los pocos ejemplares de clases escasas presentes en el entrenamiento; al enfrentarse a registros nuevos de estas mismas clases en el conjunto de prueba, el modelo pierde capacidad predictiva, hundiendo el promedio global.

10.2. Optimización de Hiperparámetros

Durante la experimentación, se observó que los modelos tendían al sobreajuste de forma persistente. La búsqueda de hiperparámetros realizada de manera empírica permitió mitigar levemente esta tendencia; sin embargo, no fue suficiente para eliminar la brecha de generalización.

Para futuros trabajos, la búsqueda empírica podría ser sustituida por métodos de optimización formal como **Grid Search CV** o **Randomized Search CV**. Estas estrategias, al evaluar sistemáticamente el espacio de búsqueda mediante validación cruzada, podrían identificar configuraciones que penalicen con mayor rigor la complejidad de los modelos, favoreciendo arquitecturas más robustas y con mejor capacidad de respuesta ante datos no conocidos.

10.3. Estrategias de Mitigación Sugeridas

Para reducir el impacto del desbalance y mejorar la capacidad de generalización, se proponen las siguientes estrategias técnicas:

- **Técnicas de Remuestreo:** Implementar *Synthetic Minority Over-sampling Technique* (SMOTE) para generar ejemplos sintéticos de las clases con bajo soporte o *Random Under Sampling* para equilibrar la clase dominante.
- **Agrupación de Clases (Re-etiquetado):** Consolidar clases minoritarias con características similares en categorías macro. Por ejemplo, agrupar las distintas variantes de *Web Attack* (Brute Force, XSS, Sql Injection) en una única clase denominada *Web Attacks*. Esto incrementaría el soporte de la clase resultante, facilitando la detección de patrones comunes y reduciendo la complejidad del espacio de búsqueda para el modelo.
- **Ajuste de Pesos de Clase (Cost-Sensitive Learning):** Configurar los algoritmos para asignar un peso mayor a los errores cometidos en las clases minoritarias durante la función de pérdida, compensando así la disparidad numérica.
- **Regularización Estricta:** Incrementar los parámetros de penalización (como *lambda* y *alpha* en XGBoost, o reducir la profundidad máxima y aumentar el número mínimo de muestras por hoja en Random Forest) para forzar al modelo a aprender patrones más globales en lugar de memorizar instancias específicas.

Finalmente, una limitación crítica detectada reside en el alto costo computacional de procesar millones de registros, lo que sugiere que cualquier optimización futura deberá considerar el uso de hardware especializado o técnicas de muestreo representativo estratificado para mantener la viabilidad técnica del estudio.

11. Conclusiones

Tras el análisis de los datos obtenidos en las 30 iteraciones del diseño experimental, se concluye que el modelo XGBoost, bajo las condiciones descritas, presenta un rendimiento estadísticamente diferente al de Random Forest. Los resultados de la prueba t de Student pareada confirman que esta diferencia es significativa, con un valor $p < 0{,}05$, lo que permite rechazar la hipótesis de igualdad de medias.

Asimismo, el modelo XGBoost exhibió un F1-Score Macro consistentemente mayor en comparación con Random Forest tanto en los conjuntos de validación como de prueba. Esta tendencia se mantuvo estable a lo largo de las ejecuciones, demostrando que la arquitectura de árboles potenciados (Boosting) logra capturar de manera más efectiva las complejidades del dataset CIC-2017 que el enfoque de ensamble por Bagging. En definitiva, la evidencia estadística sustenta que XGBoost ofrece una capacidad de generalización superior para la clasificación de tráficos maliciosos bajo la métrica evaluada.